Frame It as Cases: Work That Speaks for Itself

Week 2 — AI Fluency Assignment

Purpose

This document presents my Week 2 deliverables: framing real, hands-on cybersecurity research (OWASP Juice Shop, Kali Linux) as evidence-based case studies, in my own voice, per the Week 2 brief.

1. Voice Card

Direct, technical, precise, evidence-based.

2. Professional Bio

I test real applications for real vulnerabilities — XSS, SQL injection, and the gaps in between — using Kali Linux and OWASP Juice Shop to prove I can find what attackers find. I want to work in cybersecurity, not just study it.

3. Call to Action

Email me to talk about cybersecurity, internships, roles, security research, or opportunities to learn and contribute.

4. Before / After Writing Example

Generic AI Version

I am a passionate and results-driven cybersecurity enthusiast dedicated to leveraging cutting-edge tools to identify and mitigate vulnerabilities in dynamic environments.

Edited Version (mine)

I test real applications for real vulnerabilities — XSS, SQL injection, and the gaps in between — using Kali Linux and OWASP Juice Shop to prove I can find what attackers find.

5. Case Study: Reflected XSS — OWASP Juice Shop

The Problem

I wanted to check whether Juice Shop properly sanitized user input before rendering it back to the browser — a basic but critical trust boundary that a lot of real apps get wrong.

What I Did

I targeted the search bar on the products page. My first attempt used a standard script payload:

```
<script>alert('XSS')</script>
```

It didn't fire — that context handled script tags differently. I pivoted to an event-handler payload instead:

```
<img src=x onerror=alert('XSS')>
```

That one executed — an alert box popped up with “XSS,” confirming the browser ran my JavaScript instead of displaying it as plain text.

What Came of It

This confirmed a reflected XSS vulnerability: the app was echoing raw user input straight into the page without escaping it, meaning an attacker could run arbitrary JavaScript in another user's browser via a crafted link. The fix for this class of issue is proper output encoding and a Content Security Policy — I have not implemented or tested a fix myself yet, so I'm noting that as the honest next step, not a completed part of this work.

6. Case Study: SQL Injection — Authentication Bypass

The Problem

I wanted to check whether the login form validated user input before using it inside a SQL query — a classic and still-common failure point in authentication systems.

What I Did

I targeted the email/username field on the login form and submitted the following as input:

```
' OR 1=1--
```

It worked on the first attempt — no encoding or bypass tricks needed. I tried a couple of payload variations afterward just to see how the field behaved under different injections, but the basic bypass alone was enough to demonstrate the vulnerability.

What Came of It

The application logged me in without valid credentials, instead of returning an authentication error. That confirmed the login query was building SQL directly from unsanitized input, letting an attacker manipulate the query logic to bypass authentication entirely. The standard fix is parameterized queries (prepared statements) — again, I have not implemented or tested that fix myself, so I'm flagging it as the next step rather than claiming it as done.

7. What I'd Change Next Time

Both cases stop at confirming the vulnerability. The next iteration of this work should include a concrete impact scenario for each (e.g., what an attacker could actually do with the XSS — session token theft, credential phishing — and what account-level access the SQLi bypass grants), plus an attempted fix and retest. That would move this from "found it" to "found it, understood the blast radius, and verified the fix."